



В этом домашнем задании вам предстоит самостоятельно решить задачу классификации текстов на основе семинарского кода. Мы будем использовать датасет [ag_news](#). Это датасет для классификации новостей на 4 темы: "World", "Sports", "Business", "Sci/Tech".

Установим модуль datasets, чтобы нам проще было работать с данными.

```
# !pip install datasets  
  
# import os  
# os.environ['CUDA_LAUNCH_BLOCKING'] = '1'
```

Импорт необходимых библиотек

Постараемся зафиксировать все возможные источники случайности

```
import os  
import random  
import numpy as np  
import torch  
import sys  
  
def set_seed(seed=42):  
    """  
    Фиксация источников случайности  
    """  
    random.seed(seed)  
    np.random.seed(seed)  
    torch.manual_seed(seed)
```

```

if torch.cuda.is_available():
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

# Детерминированные алгоритмы
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

# Фиксация хэшей Python
os.environ['PYTHONHASHSEED'] = str(seed)

# Для CUDA
os.environ['CUBLAS_WORKSPACE_CONFIG'] = ':16:8'
# os.environ['CUBLAS_WORKSPACE_CONFIG'] = ':4096:8'

# Фиксация случайности
SEED = 42
set_seed(SEED)

# Фиксация случайности для генератора в Даталоэдере
g = torch.Generator().manual_seed(SEED)

import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import datasets

import numpy as np
import matplotlib.pyplot as plt

from tqdm.auto import tqdm
# from datasets import load_dataset
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split
import nltk

from collections import Counter
from typing import List
import string

import seaborn
seaborn.set_theme(palette='summer')

# nltk.download('punkt_tab')

device = 'cuda' if torch.cuda.is_available() else 'cpu'
device
'cuda'

```

Подготовка данных

Для вашего удобства, мы привели код обработки датасета в ноутбуке. Ваша задача --- обучить модель, которая получит максимальное возможное качество на тестовой части.

```
# Загрузим датасет
dataset = datasets.load_dataset('ag_news')
```

Как и в семинаре, выполним следующие шаги:

- Составим словарь
- Создадим класс WordDataset
- Выделим обучающую и тестовую часть, создадим DataLoader-ы.

```
words = Counter()

for example in tqdm(dataset['train']['text']):
    # Приводим к нижнему регистру и убираем пунктуацию
    prcessed_text = example.lower().translate(
        str.maketrans('', '', string.punctuation))

    for word in word_tokenize(prcessed_text):
        words[word] += 1

vocab = set(['<unk>', '<bos>', '<eos>', '<pad>'])
counter_threshold = 25

for char, cnt in words.items():
    if cnt > counter_threshold:
        vocab.add(char)

print(f'Размер словаря: {len(vocab)}')

word2ind = {char: i for i, char in enumerate(vocab)}
ind2word = {i: char for char, i in word2ind.items()}

{"model_id": "0ee23bf87af241c991d044e6b03a48cf", "version_major": 2, "version_minor": 0}

Размер словаря: 11842

class WordDataset:
    def __init__(self, sentences):
        self.data = sentences
        self.unk_id = word2ind['<unk>']
        self.bos_id = word2ind['<bos>']
        self.eos_id = word2ind['<eos>']
        self.pad_id = word2ind['<pad>']

    def __getitem__(self, idx: int) -> List[int]:
```

```

        processed_text = self.data[idx]['text'].lower().translate(
            str.maketrans('', '', string.punctuation))
        tokenized_sentence = [self.bos_id]
        tokenized_sentence += [
            word2ind.get(word, self.unk_id) for word in
word_tokenize(processed_text)
        ]
        tokenized_sentence += [self.eos_id]

        train_sample = {
            "text": tokenized_sentence,
            "label": self.data[idx]['label']
        }

        return train_sample

def __len__(self) -> int:
    return len(self.data)

def collate_fn_with_padding(
    input_batch: List[List[int]], pad_id=word2ind['<pad>'],
max_len=256) -> torch.Tensor:
    seq_lens = [len(x['text']) for x in input_batch]
    max_seq_len = min(max(seq_lens), max_len)

    new_batch = []
    for sequence in input_batch:
        sequence['text'] = sequence['text'][:max_seq_len]
        for _ in range(max_seq_len - len(sequence['text'])):
            sequence['text'].append(pad_id)

        new_batch.append(sequence['text'])

    sequences = torch.LongTensor(new_batch).to(device)
    labels = torch.LongTensor([x['label'] for x in
input_batch]).to(device)

    new_batch = {
        'input_ids': sequences,
        'label': labels
    }

    return new_batch

train_dataset = WordDataset(dataset['train'])

np.random.seed(42)
idx = np.random.choice(np.arange(len(dataset['test'])), 5000)
eval_dataset = WordDataset(dataset['test'].select(idx))

```

```

batch_size = 32
train_dataloader = DataLoader(
    train_dataset, shuffle=True, collate_fn=collate_fn_with_padding,
    batch_size=batch_size,
    generator=g
)

eval_dataloader = DataLoader(
    eval_dataset, shuffle=False, collate_fn=collate_fn_with_padding,
    batch_size=batch_size)

```

Постановка задачи

Ваша задача -- получить максимальное возможное accuracy на `eval_dataloader`. Ниже приведена функция, которую вам необходимо запустить для обученной модели, чтобы вычислить качество её работы.

```

def evaluate(model, eval_dataloader) -> float:
    """
    Calculate accuracy on validation dataloader.
    """

    predictions = []
    target = []
    with torch.no_grad():
        for batch in eval_dataloader:
            logits = model(batch['input_ids'])
            predictions.append(logits.argmax(dim=1))
            target.append(batch['label'])

    predictions = torch.cat(predictions)
    target = torch.cat(target)
    accuracy = (predictions == target).float().mean().item()

    return accuracy

```

Ход работы

Оценка за домашнее задание складывается из четырех частей:

Запуск базовой модели с семинара на новом датасете (1 балл)

На семинаре мы создали модель, которая дает на нашей задаче довольно высокое качество. Ваша цель --- обучить ее и вычислить score, который затем можно будет использовать в качестве бейзлайна.

В модели появится одно важное изменение: количество классов теперь равно не 2, а 4. Обратите на это внимание и найдите, что в коде создания модели нужно модифицировать, чтобы учесть это различие.

Base model

```
class CharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4,    # 4 класса
        aggregation_type: str = 'max'
    ):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.RNN(hidden_dim, hidden_dim, batch_first=True)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

        self.aggregation_type = aggregation_type

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch) # [batch_size,
        seq_len, hidden_dim]
        output, _ = self.rnn(embeddings) # [batch_size, seq_len,
        hidden_dim]

        if self.aggregation_type == 'max':
            output = output.max(dim=1)[0] # [batch_size, hidden_dim]
        elif self.aggregation_type == 'mean':
            output = output.mean(dim=1) # [batch_size, hidden_dim]
        else:
            raise ValueError('Invalid aggregation_type')

        output = self.dropout(self.linear(self.non_lin(output))) #
        [batch_size, hidden_dim]
        prediction = self.projection(self.non_lin(output)) #
        [batch_size, num_classes]

        return prediction

model = CharLM(hidden_dim=256, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

num_epoch = 7
eval_steps = len(train_dataloader) // 2

losses_type = {}
acc_type = {}

for aggregation_type in ['max', 'mean']:
```

```

print(f'Starting training for agg type: {aggregation_type}')
losses = []
acc = []

model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=aggregation_type).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

for epoch in range(num_epoch):
    epoch_losses = []
    model.train()
    for i, batch in enumerate(tqdm(train_dataloader,
desc=f'Training epoch {epoch}:')):
        optimizer.zero_grad()
        logits = model(batch['input_ids'])
        loss = criterion(logits, batch['label'])
        loss.backward()
        optimizer.step()

        epoch_losses.append(loss.item())
        if i % eval_steps == 0:
            model.eval()
            acc.append(evaluate(model, eval_dataloader)) # +
            model.train()

    losses.append(sum(epoch_losses) / len(epoch_losses))

losses_type[aggregation_type] = losses
acc_type[aggregation_type] = acc

```

Starting training for agg type: max

```

{"model_id": "302c779a273c4e1995b4e38d39f66279", "version_major": 2, "version_minor": 0}

{"model_id": "464e0e8c40614568ab421952ec6b1a6b", "version_major": 2, "version_minor": 0}

{"model_id": "c13e6fd4d2c74c57962ae6d79e9f742b", "version_major": 2, "version_minor": 0}

{"model_id": "ffcc484843a44e72b8e78c5c36c84046", "version_major": 2, "version_minor": 0}

{"model_id": "ef2d53af6aaa441a993829724b54a995", "version_major": 2, "version_minor": 0}

{"model_id": "c52a1ae1411e4115b8ac21ea3c6169fc", "version_major": 2, "version_minor": 0}

```

```
{"model_id": "36cc7da1f53647378fb55997274472f6", "version_major": 2, "version_minor": 0}
```

Starting training for agg type: mean

```
{"model_id": "13c1cfc9268c42428aa67bf3b89fb64b", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "503eca2cc09e4d00af0cc40bed8ea086", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "aec8d6a4ed664437adbee4d3e145a446", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "09643557151c4829a7853a31f88f1681", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "d95dbf9c890d42c58ea865eee373ecbd", "version_major": 2, "version_minor": 0}
```

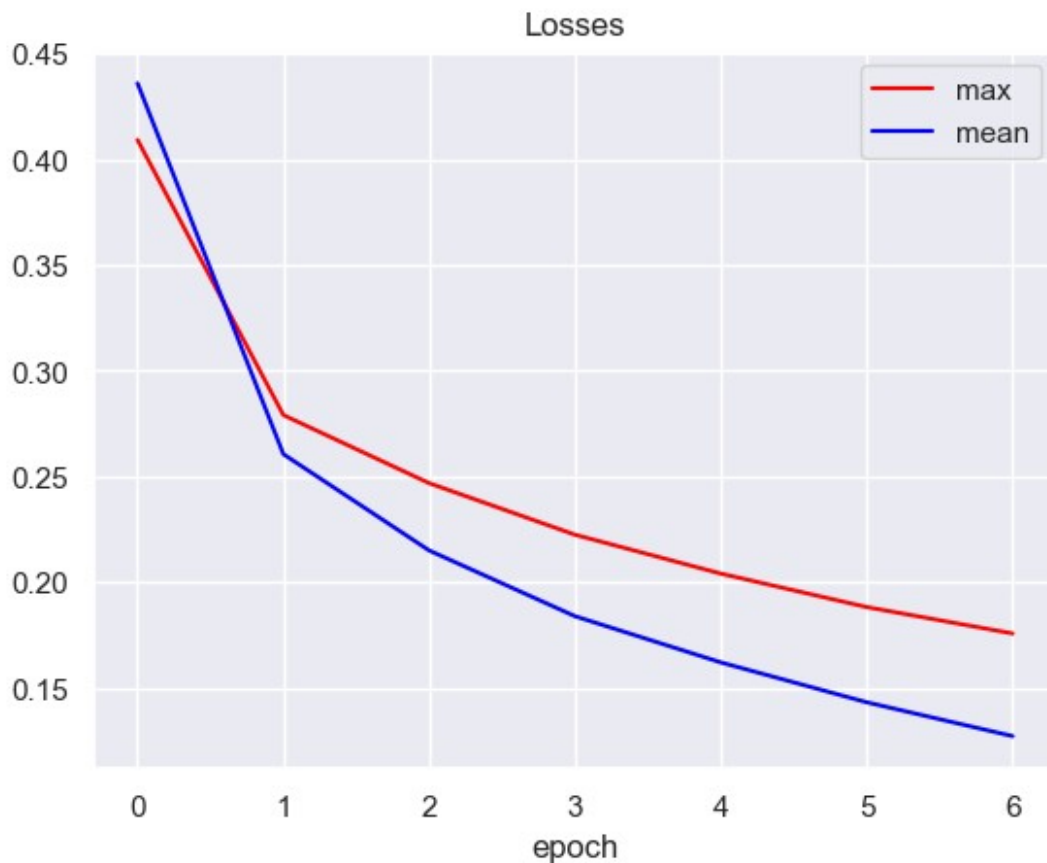
```
{"model_id": "b579494b627d43d993f9784fb5fdfae9", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "2a63614d51ae4a0e8bdee2930add889", "version_major": 2, "version_minor": 0}
```

```
for (name, values), color in zip(losses_type.items(), ['red',  
'blue']):  
    plt.plot(np.arange(len(losses_type[name])), losses_type[name],  
             color=color, label=name)
```

```
plt.title('Losses')  
plt.xlabel('epoch')  
plt.legend()
```

```
plt.show();
```

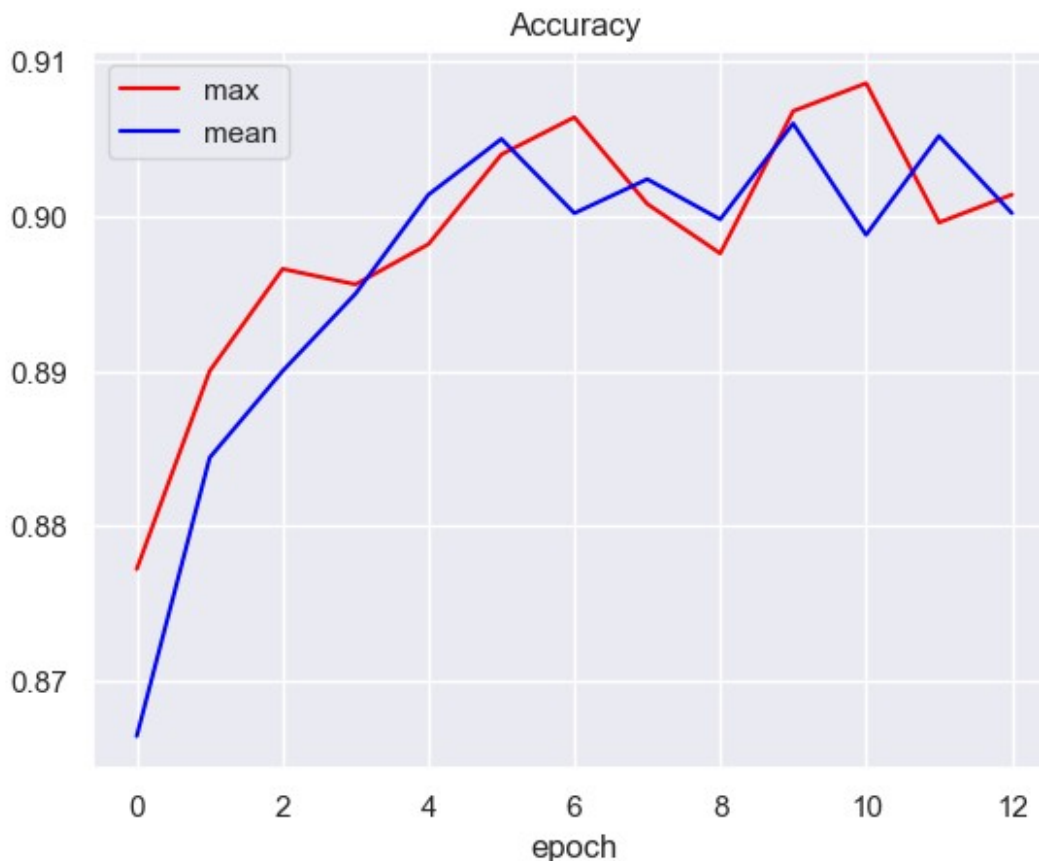


```
for (name, values), color in zip(losses_type.items(), ['red',
'blue']):
    plt.plot(np.arange(len(acc_type[name][1:])), acc_type[name][1:],
color=color, label=name)
    print(f'Лучшая accuracy для подхода {name}: {(max(acc_type[name])
* 100):.2f}')
```

```
plt.title('Accuracy')
plt.xlabel('epoch')
plt.legend()

plt.show();
```

Лучшая accuracy для подхода max: 90.86
Лучшая accuracy для подхода mean: 90.60



Проведение экспериментов по улучшению модели (2 балла за каждый эксперимент)

Чтобы улучшить качество базовой модели, можно попробовать различные идеи экспериментов. Каждый выполненный эксперимент будет оцениваться в 2 балла. Для получения полного балла за этот пункт вам необходимо выполнить по крайней мере 2 эксперимента. Не расстраивайтесь, если какой-то эксперимент не дал вам прироста к качеству: он все равно зачтется, если выполнен корректно.

Вот несколько идей экспериментов:

- **Модель RNN.** Попробуйте другие нейросетевые модели --- LSTM и GRU. Мы советуем обратить внимание на [GRU](#), так как интерфейс этого класса ничем не отличается от обычной Vanilla RNN, которую мы использовали на семинаре.
- **Увеличение количества рекуррентных слоев модели.** Это можно сделать с помощью параметра `num_layers` в классе `nn.RNN`. В такой модели выходы первой RNN передаются в качестве входов второй RNN и так далее.
- **Изменение архитектуры после применения RNN.** В базовой модели используется агрегация со всех эмбеддингов. Возможно, вы захотите конкатенировать результат агрегации и эмбеддинг с последнего токена.
- **Подбор гиперпараметров и обучение до сходимости.** Возможно, для получения более высокого качества просто необходимо увеличить количество эпох обучения

нейросети, а также попробовать различные гиперпараметры: размер словаря, `dropout_rate`, `hidden_dim`.

Обратите внимание, что главное правило проведения экспериментов --- необходимо совершать одно архитектурное изменение в одном эксперименте. Если вы совершите несколько изменений, то будет неясно, какое именно из изменений дало прирост к качеству.

Для сохранения результатов моделей (типов и их модификаций) и их последующего анализа создадим датасет.

Таблица для сравнения моделей:

```
import pandas as pd

# Значения Loss и Accuracy базовой модели RNN
base_max_loss = losses_type['max']
base_max_acc = acc_type['max']

base_mean_loss = losses_type['mean']
base_mean_acc = acc_type['mean']

# Сбор статистик для моделей
df_stats = pd.DataFrame(columns=['Model', 'Min_loss', 'Max_accuracy',
                                 'Diff_accuracy'])

loss_base_max = min(base_max_loss)
acc_base_max = max(base_max_acc)

loss_base_mean = min(base_mean_loss)
acc_base_mean = max(base_mean_acc)

data_to_add = [
    ('base_mean', loss_base_mean, acc_base_mean, acc_base_mean -
acc_base_mean),
    ('base_max', loss_base_max, acc_base_max, acc_base_max -
acc_base_mean)
]

for i, (model, min_loss, max_accuracy, diff_accuracy) in
enumerate(data_to_add):
    df_stats.loc[i] = [model, min_loss, max_accuracy, diff_accuracy]

display(df_stats)
```

	Model	Min_loss	Max_accuracy	Diff_accuracy
0	base_mean	0.127081	0.9060	0.0000
1	base_max	0.175645	0.9086	0.0026

Для обучения и сравнения экспериментальной модели с базовой моделью RNN (mean подход) напомним функцию. При агрегации max в моделях сравниваем модели с base

моделью с агрегацией max, в остальных случаях сравниваем с лучшей метрикой accuracy в base модели - mean.

```
from IPython.display import clear_output

# Функция для тренировки модели и сравнения текущей модели с базовой
# моделью RNN
def train_model(model, criterion, optimizer,
                model_name='current',
                aggregation_type='mean',
                plot=True,
                num_epoch=7,
                base_mean_loss=base_mean_loss,
                base_mean_acc=base_mean_acc,
                train_loader=train_data_loader,
                test_loader=eval_data_loader):

    eval_steps = len(train_loader) // 2

    total_loss = []
    total_acc = []

    print(f'\n{model_name} модель')

    for epoch in tqdm(range(num_epoch), desc=f'Training
{model_name}'):
        epoch_train_loss = 0

        model.train()
        for i, batch in enumerate(train_loader):    # for i, batch in
enumerate(tqdm(train_loader)):
            optimizer.zero_grad()
            logits = model(batch['input_ids'])
            loss = criterion(logits, batch['label'])
            loss.backward()
            optimizer.step()
            epoch_train_loss += loss.item()

            if i % eval_steps == 0:
                model.eval()
                total_acc.append(evaluate(model, test_loader))
                model.train()

        total_loss.append(epoch_train_loss / len(train_loader))

    if aggregation_type == 'max':
        base_loss = base_max_loss
        base_acc = base_max_acc
    else:
        base_loss = base_mean_loss
```

```

        base_acc = base_mean_acc

        # Отрисовка графиков
        if plot:

            fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(15,
6))

            axes[0].plot(base_loss[:epoch+1], color='b', marker='o',
markersize=4, label='base')
            axes[0].plot(total_loss, color='r', marker='o',
markersize=4, label=model_name)

            axes[0].set_title('Losses')
            axes[0].set_xlabel('epoch')
            axes[0].set_ylabel('loss')
            axes[0].grid(True)
            axes[0].legend()

            axes[1].plot(base_acc[:epoch+1]*2, color='b',
marker='o', markersize=4, label='base')
            axes[1].plot(total_acc, color='r', marker='o',
markersize=4, label=model_name)

            axes[1].set_title('Accuracy')
            axes[1].set_xlabel('epoch')
            axes[1].set_ylabel('accuracy')
            axes[1].set_ylim([0.87, 0.93])
            axes[1].grid(True)
            axes[1].legend()

            clear_output(wait=True)
            plt.show()

        # Наполнение таблицы
        curr_model_loss = min(total_loss)
        curr_model_acc = max(total_acc)
        diff_accuracy = curr_model_acc - max(base_acc)

        data_to_add = (model_name, curr_model_loss, curr_model_acc,
diff_accuracy)
        if model_name in df_stats['Model'].values:
            index = df_stats[df_stats['Model'] == model_name].index
        else:
            index = len(df_stats)
        df_stats.loc[index] = data_to_add

        print(f'Accuracy = {max(total_acc)}')
        print(('Хуже', 'Лучше')[curr_model_acc > acc_base_mean], 'на',

```

```
abs(diff_accuracy))
    return total_loss, total_acc
```

Base Tuned модель

Добавим несколько вариантов агрегаций (aggregation_type), возможность изменения количества слоев (num_layers) и вероятность отключения нейронов (dropout_rate)

```
class CharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4,    # 4 класса
        aggregation_type: str = 'max',
        num_layers=1,
        dropout_rate=0
    ):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)

        if num_layers > 1:
            rnn_dropout = dropout_rate
        else:
            rnn_dropout = 0.0

        self.rnn = nn.RNN(hidden_dim, hidden_dim, batch_first=True,
            num_layers=num_layers, dropout=rnn_dropout)

        if aggregation_type == 'mean_last_concat':
            agg_dim = hidden_dim * 2
        elif aggregation_type == 'multi_concat':
            agg_dim = hidden_dim * 4
        else:
            agg_dim = hidden_dim

        self.linear = nn.Linear(agg_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=dropout_rate) if dropout_rate > 0
    else nn.Identity()

    self.aggregation_type = aggregation_type

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch) # [batch_size,
        seq_len, hidden_dim]
        output, _ = self.rnn(embeddings) # [batch_size, seq_len,
        hidden_dim]
        # Max
```

```

        if self.aggregation_type == 'max':
            output = output.max(dim=1)[0] #[batch_size, hidden_dim]
        # Mean
        elif self.aggregation_type == 'mean':
            output = output.mean(dim=1) #[batch_size, hidden_dim]
        # Sum
        elif self.aggregation_type == 'sum':
            output = output.sum(dim=1) # [batch_size, hidden_dim]
        # Max с нормализацией
        elif self.aggregation_type == 'max_norm':
            output = output.max(dim=1)[0]
            output = torch.nn.functional.normalize(output, p=2, dim=1)
        # [batch_size, hidden_dim]
        # Mean с нормализацией
        elif self.aggregation_type == 'mean_norm':
            output = output.mean(dim=1)
            output = torch.nn.functional.normalize(output, p=2, dim=1)
        # [batch_size, hidden_dim]
        # Mean_last_concat
        elif self.aggregation_type == 'mean_last_concat':
            mean_pool = output.mean(dim=1)
            last = output[:, -1, :]
            output = torch.cat([mean_pool, last], dim=1) #
        [batch_size, 2*hidden_dim]
        # Multi_concat
        elif self.aggregation_type == 'multi_concat':
            max_pool = output.max(dim=1)[0]
            mean_pool = output.mean(dim=1)
            first = output[:, 0, :]
            last = output[:, -1, :]
            output = torch.cat([max_pool, mean_pool, first, last],
            dim=1) # [batch_size, 4*hidden_dim]
        else:
            raise ValueError('Invalid aggregation_type')

        output = self.dropout(self.linear(self.non_lin(output))) #
        [batch_size, hidden_dim]
        prediction = self.projection(self.non_lin(output)) #
        [batch_size, num_classes]

        return prediction

name_model = 'base'

```

Изменение архитектуры после применения RNN

Изменение aggregation_type

```

aggregation_types = ['max', 'mean', 'sum', 'max_norm', 'mean_norm',
'mean_last_concat', 'multi_concat']
model_names = ['base_' + agg_type for agg_type in aggregation_types]

for name_model, agg_type in zip(model_names, aggregation_types):
    set_seed(SEED)
    model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
    criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
    optimizer = torch.optim.Adam(model.parameters())

    model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=name_model, plot=False, num_epoch=7)

```

base_max модель

```

{"model_id": "1a5966a8d7da43e2990b149988dd401d", "version_major": 2, "version_minor": 0}

```

Accuracy = 0.9081999659538269
Лучше на 0.002200007438659668

base_mean модель

```

{"model_id": "0e57afbd70114d848693c26e793e7418", "version_major": 2, "version_minor": 0}

```

Accuracy = 0.9079999923706055
Лучше на 0.0020000338554382324

base_sum модель

```

{"model_id": "b72db8a9d2b3481793b16f63f5ff2cbf", "version_major": 2, "version_minor": 0}

```

Accuracy = 0.8999999761581421
Хуже на 0.0059999823570251465

base_max_norm модель

```

{"model_id": "df067ee0199a4a97a2daa50f363ea8f8", "version_major": 2, "version_minor": 0}

```

Accuracy = 0.901199996471405
Хуже на 0.004799962043762207

base_mean_norm модель

```

{"model_id": "c0a01e4dcfd44d00b174b28db59e60ed", "version_major": 2, "version_minor": 0}

```

```
Accuracy = 0.9091999530792236  
Лучше на 0.0031999945640563965
```

base_mean_last_concat модель

```
{"model_id": "5884dec93d954c3fb33e96f9f2fd0e36", "version_major": 2, "version_minor": 0}
```

```
Accuracy = 0.9097999930381775  
Лучше на 0.003800034523010254
```

base_multi_concat модель

```
{"model_id": "c229fe32439d4ea083685680184b9daf", "version_major": 2, "version_minor": 0}
```

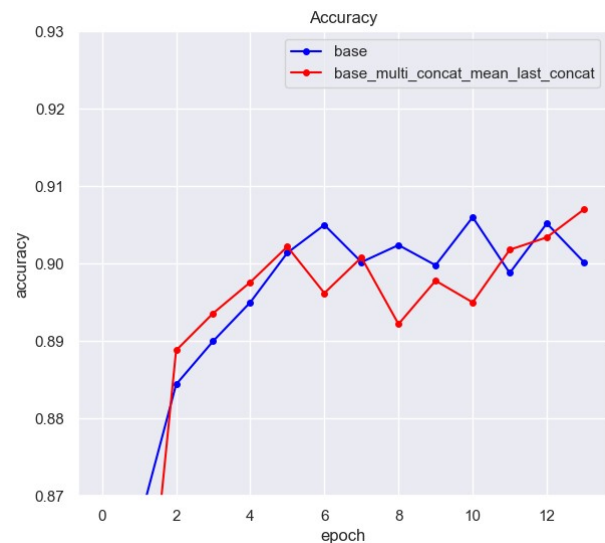
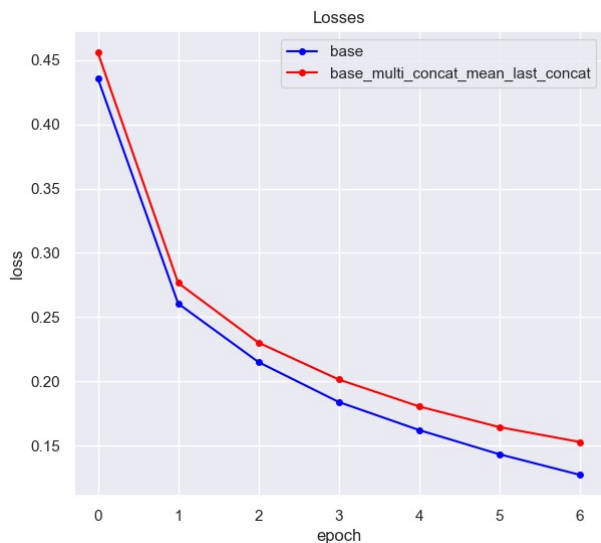
```
Accuracy = 0.9007999897003174  
Хуже на 0.0051999688148498535
```

```
aggregation_types = ['sum', 'max_norm', 'mean_norm',  
                     'mean_last_concat', 'multi_concat']
```

```
agg_type = 'mean_last_concat'
```

```
model = CharLM(hidden_dim=256, vocab_size=len(vocab),  
               aggregation_type=agg_type).to(device)  
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])  
optimizer = torch.optim.Adam(model.parameters())
```

```
model_loss, model_acc = train_model(model, criterion, optimizer,  
                                   model_name=f'{name_model}_{agg_type}')
```



Accuracy = 0.9070000052452087
Лучше на 0.001000046730041504

```
# aggregation_types = ['sum', 'max_norm', 'mean_norm',  
                        'mean_last_concat', 'multi_concat']  
  
# agg_type = 'sum'  
  
# model = CharLM(hidden_dim=256, vocab_size=len(vocab),  
                  aggregation_type=agg_type).to(device)  
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])  
# optimizer = torch.optim.Adam(model.parameters())  
  
# model_loss, model_acc = train_model(model, criterion, optimizer,  
model_name=f'{name_model}_{agg_type}')  
  
# aggregation_types = ['sum', 'max_norm', 'mean_norm',  
                        'mean_last_concat', 'multi_concat']  
  
# agg_type = 'max_norm'  
  
# model = CharLM(hidden_dim=256, vocab_size=len(vocab),  
                  aggregation_type=agg_type).to(device)  
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])  
# optimizer = torch.optim.Adam(model.parameters())  
  
# model_loss, model_acc = train_model(model, criterion, optimizer,  
model_name=f'{name_model}_{agg_type}')  
  
# aggregation_types = ['sum', 'max_norm', 'mean_norm',  
                        'mean_last_concat', 'multi_concat']  
  
# agg_type = 'mean_norm'  
  
# model = CharLM(hidden_dim=256, vocab_size=len(vocab),  
                  aggregation_type=agg_type).to(device)  
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])  
# optimizer = torch.optim.Adam(model.parameters())  
  
# model_loss, model_acc = train_model(model, criterion, optimizer,  
model_name=f'{name_model}_{agg_type}')  
  
# aggregation_types = ['sum', 'max_norm', 'mean_norm',  
                        'mean_last_concat', 'multi_concat']  
  
# agg_type = 'multi_concat'  
  
# model = CharLM(hidden_dim=256, vocab_size=len(vocab),  
                  aggregation_type=agg_type).to(device)  
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])  
# optimizer = torch.optim.Adam(model.parameters())
```

```
# model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')
```

Увеличение количества рекуррентных слоев модели

Изменение num_layers и dropout_rate (в связке между новыми слоями)

С помощью itertools.product сможем перебрать все возможные комбинации заданных гиперпараметров (настроек) модели

```
import itertools

num_layers_list = [1, 2, 3]
dropout_rates = [0.0, 0.3]
agg_type = 'mean'
num_epoch = 7

for num_layers, dropout_rate in itertools.product(num_layers_list,
dropout_rates):
    set_seed(SEED)
    model = CharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type, num_layers=num_layers,
dropout_rate=dropout_rate).to(device)
    criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
    optimizer = torch.optim.Adam(model.parameters())

    model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}',
aggregation_type=agg_type,
plot=False, num_epoch=num_epoch)
```

base_multi_concat_mean_nl1_drop0.0 модель

```
{"model_id": "71be142b668a4946acc21509b7609cf3", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9088000059127808
Лучше на 0.0028000473976135254

base_multi_concat_mean_nl1_drop0.3 модель

```
{"model_id": "1876bd728c47460888beea77b314b0f7", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9067999720573425
Лучше на 0.000800013542175293

base_multi_concat_mean_nl2_drop0.0 модель

```
{"model_id": "46a4595ba4ab49f99c6da01e939d4f85", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9081999659538269
Лучше на 0.002200007438659668

base_multi_concat_mean_nl2_drop0.3 модель

```
{"model_id": "5b902c9dca504749b45e094245b17785", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9070000052452087
Лучше на 0.001000046730041504

base_multi_concat_mean_nl3_drop0.0 модель

```
{"model_id": "6ba8ea29bde54a17961a67e9caeead05", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9043999910354614
Хуже на 0.0015999674797058105

base_multi_concat_mean_nl3_drop0.3 модель

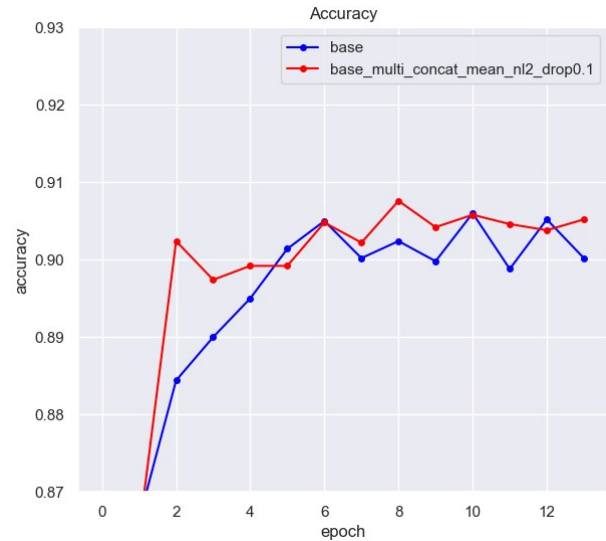
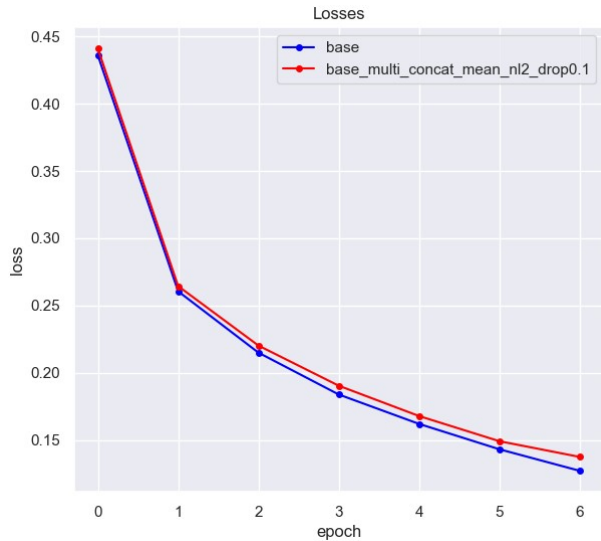
```
{"model_id": "db32c37347ca480f9e4afacffdbed09c", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9037999510765076
Хуже на 0.002200007438659668

```
num_layers = 2  
dropout_rate = 0.1  
agg_type = 'mean'
```

```
model = CharLM(hidden_dim=256, vocab_size=len(vocab),  
aggregation_type=agg_type, num_layers=num_layers,  
dropout_rate=dropout_rate).to(device)  
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])  
optimizer = torch.optim.Adam(model.parameters())
```

```
model_loss, model_acc = train_model(model, criterion, optimizer,  
model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}'  
)
```



Accuracy = 0.9075999855995178
 Лучше на 0.001600027084350586

```
# num_layers = 2
# dropout_rate = 0
# agg_type = 'mean'
```

```
# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
# aggregation_type=agg_type, num_layers=num_layers,
# dropout_rate=dropout_rate).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())
```

```
# model_loss, model_acc = train_model(model, criterion, optimizer,
# model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}'
# )
```

```
# num_layers = 2
# dropout_rate = 0.2
# agg_type = 'mean'
```

```
# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
# aggregation_type=agg_type, num_layers=num_layers,
# dropout_rate=dropout_rate).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())
```

```
# model_loss, model_acc = train_model(model, criterion, optimizer,
# model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}'
# )
```

```
# num_layers = 3
# dropout_rate = 0
```

```

# agg_type = 'mean'

# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
# aggregation_type=agg_type, num_layers=num_layers,
# dropout_rate=dropout_rate).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())

# model_loss, model_acc = train_model(model, criterion, optimizer,
# model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}'
# ')

# num_layers = 3
# dropout_rate = 0.3
# agg_type = 'mean'

# model = CharLM(hidden_dim=256, vocab_size=len(vocab),
# aggregation_type=agg_type, num_layers=num_layers,
# dropout_rate=dropout_rate).to(device)
# criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(model.parameters())

# model_loss, model_acc = train_model(model, criterion, optimizer,
# model_name=f'{name_model}_{agg_type}_nl{num_layers}_drop{dropout_rate}'
# ')

```

LSTM модель

```

class LSTMCharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4,
        aggregation_type: str = 'max',
        num_layers=1
    ):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim, batch_first=True,
num_layers=num_layers)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

        self.aggregation_type = aggregation_type

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch) # [batch_size,

```

```

seq_len, hidden_dim]
    output, _ = self.rnn(embeddings) # [batch_size, seq_len,
hidden_dim]

    if self.aggregation_type == 'max':
        output = output.max(dim=1)[0] #[batch_size, hidden_dim]
    elif self.aggregation_type == 'mean':
        output = output.mean(dim=1) #[batch_size, hidden_dim]
    else:
        raise ValueError('Invalid aggregation_type')

    output = self.dropout(self.linear(self.non_lin(output))) #
[batch_size, hidden_dim]
    prediction = self.projection(self.non_lin(output)) #
[batch_size, num_classes]

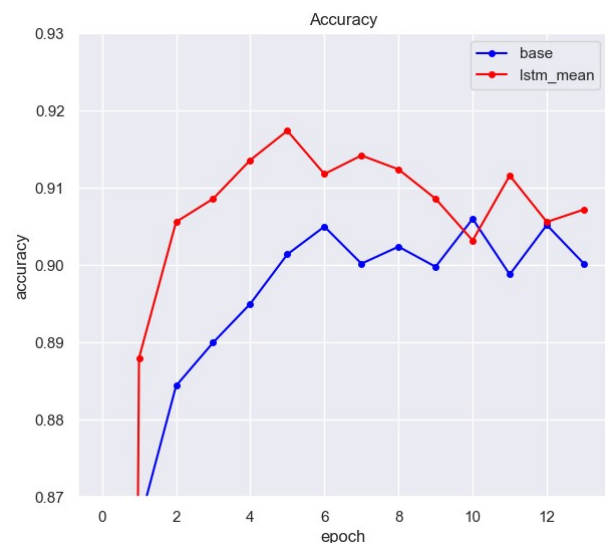
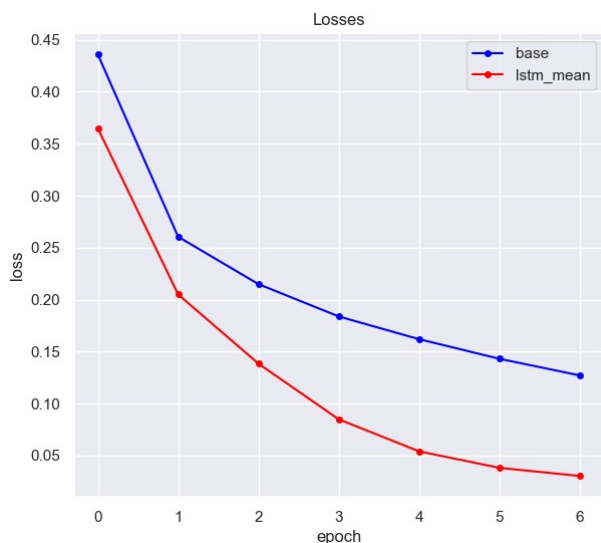
    return prediction

name_model = 'lstm'
agg_type = 'mean'

model = LSTMCharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')

```



Accuracy = 0.9174000024795532
Лучше на 0.011400043964385986

GRU модель

```
class GRUCharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4,
        aggregation_type: str = 'max',
        num_layers=1
    ):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.GRU(hidden_dim, hidden_dim, batch_first=True,
num_layers=num_layers)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

        self.aggregation_type = aggregation_type

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch) # [batch_size,
seq_len, hidden_dim]
        output, _ = self.rnn(embeddings) # [batch_size, seq_len,
hidden_dim]

        if self.aggregation_type == 'max':
            output = output.max(dim=1)[0] #[batch_size, hidden_dim]
        elif self.aggregation_type == 'mean':
            output = output.mean(dim=1) #[batch_size, hidden_dim]
        else:
            raise ValueError('Invalid aggregation_type')

        output = self.dropout(self.linear(self.non_lin(output))) #
[batch_size, hidden_dim]
        prediction = self.projection(self.non_lin(output)) #
[batch_size, num_classes]

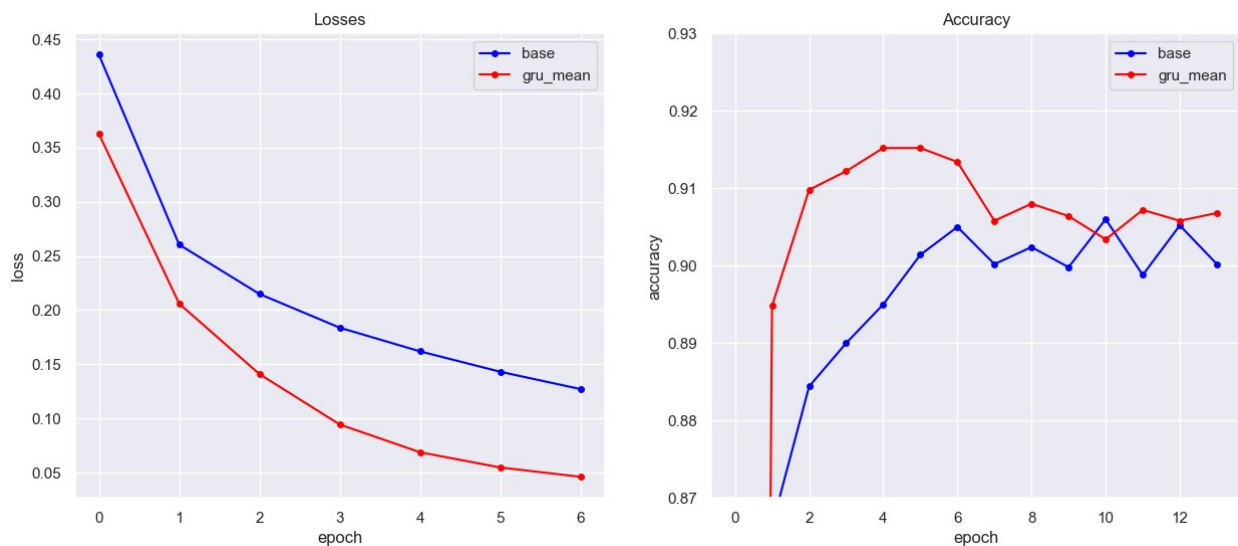
        return prediction

name_model = 'gru'

agg_type = 'mean'

model = GRUCharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())
```

```
model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')
```



Accuracy = 0.915199950408936
Лучше на 0.009200036525726318

Bidirectional RNN модель

```
class BidirectCharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4,
        aggregation_type: str = 'max',
        num_layers=1
    ):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.RNN(hidden_dim, hidden_dim, batch_first=True,
        bidirectional=True, num_layers=num_layers)
        self.linear = nn.Linear(hidden_dim * 2, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

        self.aggregation_type = aggregation_type

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch) # [batch_size,
        seq_len, hidden_dim]
```

```

        output, _ = self.rnn(embeddings) # [batch_size, seq_len,
hidden_dim]

        if self.aggregation_type == 'max':
            output = output.max(dim=1)[0] #[batch_size, hidden_dim]
        elif self.aggregation_type == 'mean':
            output = output.mean(dim=1) #[batch_size, hidden_dim]
        else:
            raise ValueError('Invalid aggregation_type')

        output = self.dropout(self.linear(self.non_lin(output))) #
[batch_size, hidden_dim]
        prediction = self.projection(self.non_lin(output)) #
[batch_size, num_classes]

        return prediction

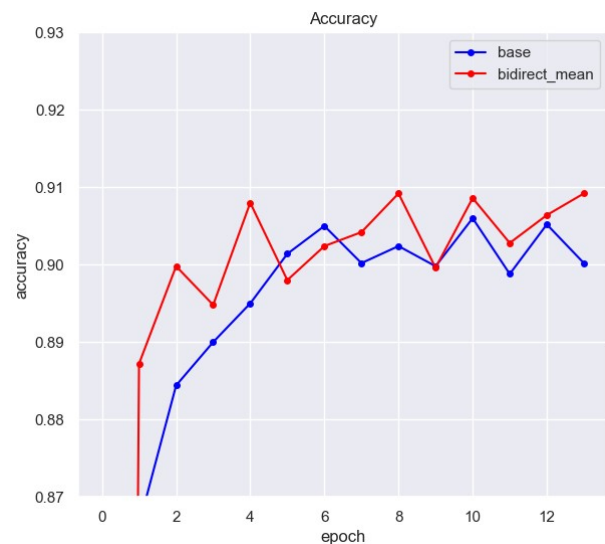
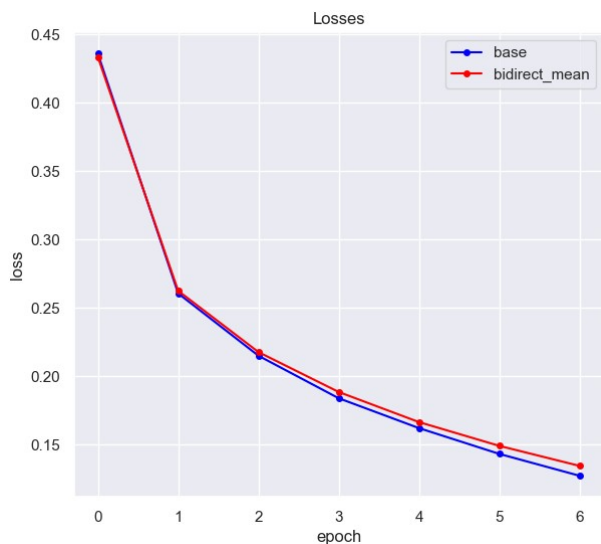
name_model = 'bidirect'

agg_type = 'mean'

model = BidirectCharLM(hidden_dim=256, vocab_size=len(vocab),
aggregation_type=agg_type).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}')

```



Accuracy = 0.9091999530792236
Лучше на 0.0031999945640563965

Подбор гиперпараметров и обучение до сходимости

Для чистоты эксперимента обучим базовую модель на 10 эпохах

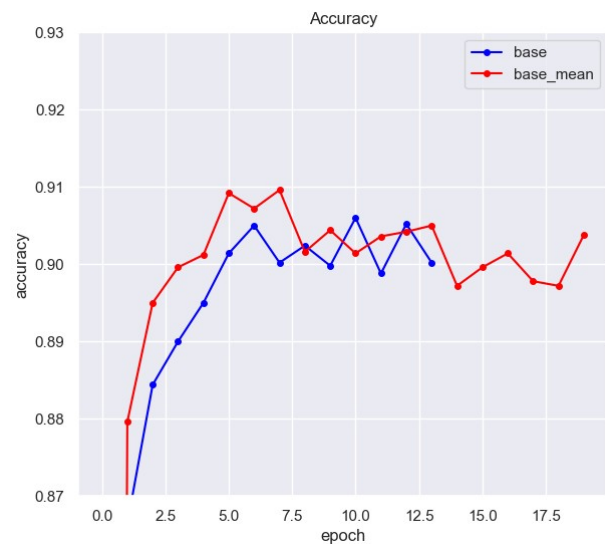
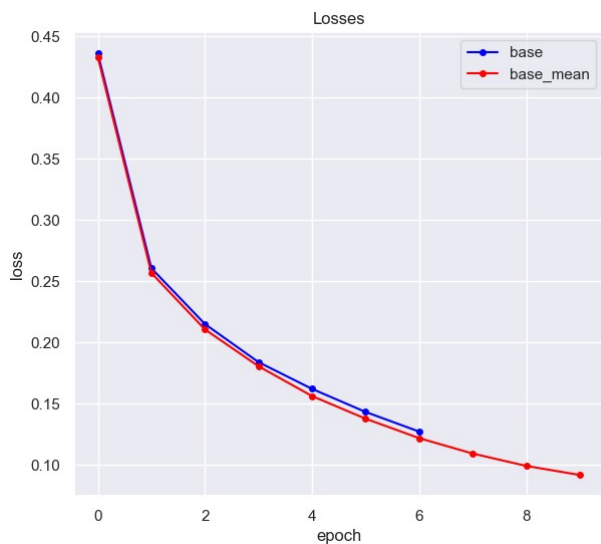
```
num_epoch = 10

name_model = 'base'

agg_type = 'mean'

model = CharLM(hidden_dim=256, vocab_size=len(vocab),
                aggregation_type=agg_type).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

base_mean_loss, base_mean_acc = train_model(model, criterion,
                                             optimizer, model_name=f'{name_model}_{agg_type}',
                                             aggregation_type=agg_type,
                                             plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
                                             base_mean_acc=base_mean_acc)
```



Accuracy = 0.9095999598503113
Лучше на 0.003600001335144043

display(df_stats)

	Model	Min_loss	Max_accuracy
Diff_accuracy			
0	base_mean	0.091620	0.9096
0.0036			
1	base_max	0.175734	0.9082
0.0022			
2	base_sum	0.287500	0.9000

-0.0060			
3	base_max_norm	0.131303	0.9012
-0.0048			
4	base_mean_norm	0.133311	0.9092
0.0032			
5	base_mean_last_concat	0.150535	0.9098
0.0038			
6	base_multi_concat	0.241421	0.9008
-0.0052			
7	base_multi_concat_mean_last_concat	0.152667	0.9070
0.0010			
8	base_multi_concat_mean_nl1_drop0.0	0.117460	0.9088
0.0028			
9	base_multi_concat_mean_nl1_drop0.3	0.140076	0.9068
0.0008			
10	base_multi_concat_mean_nl2_drop0.0	0.128997	0.9082
0.0022			
11	base_multi_concat_mean_nl2_drop0.3	0.154546	0.9070
0.0010			
12	base_multi_concat_mean_nl3_drop0.0	0.172486	0.9044
-0.0016			
13	base_multi_concat_mean_nl3_drop0.3	0.190614	0.9038
-0.0022			
14	base_multi_concat_mean_nl2_drop0.1	0.137413	0.9076
0.0016			
15	lstm_mean	0.030337	0.9174
0.0114			
16	gru_mean	0.046151	0.9152
0.0092			
17	bidirect_mean	0.134317	0.9092
0.0032			

Выберем модель с наилучшим качеством при стандартных параметрах: LSTM модель

```
class LSTMCharLM(nn.Module):
    def __init__(
        self, hidden_dim: int,
        vocab_size: int,
        num_classes: int = 4, # 4 класса
        aggregation_type: str = 'max',
        num_layers=1,
        dropout_rate=0
    ):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)

        if num_layers > 1:
            rnn_dropout = dropout_rate
        else:
```

```

        rnn_dropout = 0.0

        self.rnn = nn.LSTM(hidden_dim, hidden_dim, batch_first=True,
num_layers=num_layers, dropout=rnn_dropout)

        if aggregation_type == 'mean_last_concat':
            agg_dim = hidden_dim * 2
        else:
            agg_dim = hidden_dim

        self.linear = nn.Linear(agg_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, num_classes)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=dropout_rate) if dropout_rate > 0
else nn.Identity()

        self.aggregation_type = aggregation_type

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch) # [batch_size,
seq_len, hidden_dim]
        output, _ = self.rnn(embeddings) # [batch_size, seq_len,
hidden_dim]
        # Max
        if self.aggregation_type == 'max':
            output = output.max(dim=1)[0] #[batch_size, hidden_dim]
        # Mean
        elif self.aggregation_type == 'mean':
            output = output.mean(dim=1) #[batch_size, hidden_dim]
        # Mean_last_concat
        elif self.aggregation_type == 'mean_last_concat':
            mean_pool = output.mean(dim=1)
            last = output[:, -1, :]
            output = torch.cat([mean_pool, last], dim=1) #
[batch_size, 2*hidden_dim]
        else:
            raise ValueError('Invalid aggregation_type')

        output = self.dropout(self.linear(self.non_lin(output))) #
[batch_size, hidden_dim]
        prediction = self.projection(self.non_lin(output)) #
[batch_size, num_classes]

        return prediction

```

Для этой модели реализуем перебор гиперпараметров (настроек) модели и выберем наилучший набор гиперпараметров

```

import itertools

name_model = 'lstm'

hidden_dims = [256, 384, 512]
aggregation_types = ['mean', 'max', 'mean_last_concat']
num_layers_list = [1, 2, 3]
dropout_rates = [0.0, 0.1, 0.2, 0.3]

num_epoch = 7

best_acc = 0
best_params = {}

for hidden_dim, num_layers, dropout_rate, agg_type in
itertools.product(hidden_dims, num_layers_list, dropout_rates,
aggregation_types):
    set_seed(SEED)
    model = LSTMCharLM(hidden_dim=hidden_dim, vocab_size=len(vocab),
aggregation_type=agg_type, num_layers=num_layers,
dropout_rate=dropout_rate).to(device)
    criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
    optimizer = torch.optim.Adam(model.parameters())

    model_loss, model_acc = train_model(model, criterion, optimizer,
model_name=f'{name_model}_{agg_type}_hd{hidden_dim}_nl{num_layers}_dro
p{dropout_rate}',
                                aggregation_type=agg_type,
plot=False, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)

    curr_acc = max(model_acc)
    if curr_acc > best_acc:
        best_acc = curr_acc
        best_params = {
            'hidden_dim': hidden_dim,
            'aggregation_type': agg_type,
            'num_layers': num_layers,
            'dropout_rate': dropout_rate
        }

lstm_mean_hd256_nl1_drop0.0 модель

{"model_id": "d381551cfdc74d1bbe25343e359669d6", "version_major": 2, "vers
ion_minor": 0}

Accuracy = 0.9139999747276306
Лучше на 0.004400014877319336

lstm_max_hd256_nl1_drop0.0 модель

```

```
{"model_id": "94d0757a98b849e2ab691c51a5df00f9", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9095999598503113
Лучше на 0.0009999871253967285

lstm_mean_last_concat_hd256_nll_drop0.0 модель

```
{"model_id": "7f9d963a8c014f24aab8043ab500c9c9", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9143999814987183
Лучше на 0.004800021648406982

lstm_mean_hd256_nll_drop0.1 модель

```
{"model_id": "842ba30e716c48528e56f18f900ce15c", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9145999550819397
Лучше на 0.004999995231628418

lstm_max_hd256_nll_drop0.1 модель

```
{"model_id": "4d183595ed194585a0fe5ddbdf4680a9", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9101999998092651
Лучше на 0.001600027084350586

lstm_mean_last_concat_hd256_nll_drop0.1 модель

```
{"model_id": "8703d0d4b6154b229991bc820a606261", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9157999753952026
Лучше на 0.006200015544891357

lstm_mean_hd256_nll_drop0.2 модель

```
{"model_id": "da6bce9bca9b4af1898bdb424c3668b3", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9143999814987183
Лучше на 0.004800021648406982

lstm_max_hd256_nll_drop0.2 модель

```
{"model_id": "8c5edf1d48a242ea9b4104fea2e89024", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9120000004768372
Лучше на 0.0034000277519226074

lstm_mean_last_concat_hd256_nl1_drop0.2 модель

```
{"model_id": "3d55f85915c84c27bb2d367264395a5b", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9151999950408936
Лучше на 0.005600035190582275

lstm_mean_hd256_nl1_drop0.3 модель

```
{"model_id": "6555af396d9e48d09bfcdb4b30fee18a", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9138000011444092
Лучше на 0.0042000412940979

lstm_max_hd256_nl1_drop0.3 модель

```
{"model_id": "02b13af2764f4b53af9c769807f5d28e", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9121999740600586
Лучше на 0.003600001335144043

lstm_mean_last_concat_hd256_nl1_drop0.3 модель

```
{"model_id": "e03b8e92a23846bda167b2d7ee028c39", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9127999544143677
Лучше на 0.0031999945640563965

lstm_mean_hd256_nl2_drop0.0 модель

```
{"model_id": "c049bdb3f7304348b118fc6652be7121", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9151999950408936
Лучше на 0.005600035190582275

lstm_max_hd256_nl2_drop0.0 модель

```
{"model_id": "35bb34125f54482584d7858d40dee71c", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9143999814987183
Лучше на 0.005800008773803711

lstm_mean_last_concat_hd256_nl2_drop0.0 модель

```
{"model_id": "9fa8af94056b4c6d96690c1887945145", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9179999828338623
Лучше на 0.008400022983551025

lstm_mean_hd256_nl2_drop0.1 модель

{"model_id": "de2e387ff3154b238d63c47664121977", "version_major": 2, "version_minor": 0}

Accuracy = 0.9192000031471252
Лучше на 0.009600043296813965

lstm_max_hd256_nl2_drop0.1 модель

{"model_id": "1cab5f3955f74846837b751802f558f0", "version_major": 2, "version_minor": 0}

Accuracy = 0.9157999753952026
Лучше на 0.007200002670288086

lstm_mean_last_concat_hd256_nl2_drop0.1 модель

{"model_id": "2c67fde49d6541cf8f8f050f50121cd5", "version_major": 2, "version_minor": 0}

Accuracy = 0.9139999747276306
Лучше на 0.004400014877319336

lstm_mean_hd256_nl2_drop0.2 модель

{"model_id": "7f0e6d698ec54841843fb7784f810a72", "version_major": 2, "version_minor": 0}

Accuracy = 0.9177999496459961
Лучше на 0.008199989795684814

lstm_max_hd256_nl2_drop0.2 модель

{"model_id": "7d66d790356d498f81670fa9dcc6b0ab", "version_major": 2, "version_minor": 0}

Accuracy = 0.913599967956543
Лучше на 0.004999995231628418

lstm_mean_last_concat_hd256_nl2_drop0.2 модель

{"model_id": "376517a15486463392a89dce35c6ab11", "version_major": 2, "version_minor": 0}

Accuracy = 0.9131999611854553
Лучше на 0.003600001335144043

lstm_mean_hd256_nl2_drop0.3 модель

```
{"model_id": "2f316ed405864328b508f6a101e85831", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.917199969291687
Лучше на 0.007600009441375732

lstm_max_hd256_nl2_drop0.3 модель

```
{"model_id": "66278343f7f045d2b823053ef4f80cf5", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9163999557495117
Лучше на 0.007799983024597168

lstm_mean_last_concat_hd256_nl2_drop0.3 модель

```
{"model_id": "89fb400980554cffb0948e87bcd1bd06", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9151999950408936
Лучше на 0.005600035190582275

lstm_mean_hd256_nl3_drop0.0 модель

```
{"model_id": "79a4d912c1964afe9d02b36191bd56e3", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.913599967956543
Лучше на 0.0040000081062316895

lstm_max_hd256_nl3_drop0.0 модель

```
{"model_id": "7ac7837e096e4b929bfe6b24595d3f25", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9138000011444092
Лучше на 0.005200028419494629

lstm_mean_last_concat_hd256_nl3_drop0.0 модель

```
{"model_id": "103e56ebbf6141c6b1087be9f2e21c21", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9161999821662903
Лучше на 0.006600022315979004

lstm_mean_hd256_nl3_drop0.1 модель

```
{"model_id": "e81cfae5e6ad44bda0e9c57f4f370668", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.913599967956543
Лучше на 0.0040000081062316895

lstm_max_hd256_nl3_drop0.1 модель

```
{"model_id": "5a0c8bc71c59488db4aaa21e6e961b19", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9133999943733215

Лучше на 0.004800021648406982

lstm_mean_last_concat_hd256_nl3_drop0.1 модель

```
{"model_id": "9e9549d6930c4b71abfa0acd8082f14a", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9145999550819397

Лучше на 0.004999995231628418

lstm_mean_hd256_nl3_drop0.2 модель

```
{"model_id": "afb5a6325ee64d169856531070d8f307", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9139999747276306

Лучше на 0.004400014877319336

lstm_max_hd256_nl3_drop0.2 модель

```
{"model_id": "e7a5d98f99584ecda166fe6f4f88947e", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9127999544143677

Лучше на 0.004199981689453125

lstm_mean_last_concat_hd256_nl3_drop0.2 модель

```
{"model_id": "d975f8ee18e64006bbd91dccf9a40a46", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9159999489784241

Лучше на 0.006399989128112793

lstm_mean_hd256_nl3_drop0.3 модель

```
{"model_id": "afea5881abd1459a9a9a9cfd4cc7d970", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.911799967288971

Лучше на 0.002200007438659668

lstm_max_hd256_nl3_drop0.3 модель

```
{"model_id": "55b5e916e040433f9e84133fbf0603f2", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.914199948310852
Лучше на 0.0055999755859375

lstm_mean_last_concat_hd256_nl3_drop0.3 модель

{"model_id": "14e7839b768e4547b1530af83b8e2068", "version_major": 2, "version_minor": 0}

Accuracy = 0.9165999889373779
Лучше на 0.00700002908706665

lstm_mean_hd384_nl1_drop0.0 модель

{"model_id": "91bc2f4f1a2a419aa1134bc557cbea91", "version_major": 2, "version_minor": 0}

Accuracy = 0.9199999570846558
Лучше на 0.010399997234344482

lstm_max_hd384_nl1_drop0.0 модель

{"model_id": "f9d048bada224e0ab6a23e50fdf8898e", "version_major": 2, "version_minor": 0}

Accuracy = 0.9095999598503113
Лучше на 0.0009999871253967285

lstm_mean_last_concat_hd384_nl1_drop0.0 модель

{"model_id": "9cf6bc2e872e48bd8a683012b77b2665", "version_major": 2, "version_minor": 0}

Accuracy = 0.9195999503135681
Лучше на 0.009999990463256836

lstm_mean_hd384_nl1_drop0.1 модель

{"model_id": "da9f1da29b294fbc9de0bdb587c3b2de", "version_major": 2, "version_minor": 0}

Accuracy = 0.9156000018119812
Лучше на 0.006000041961669922

lstm_max_hd384_nl1_drop0.1 модель

{"model_id": "846f2c7be4f347fb989052957b59899b", "version_major": 2, "version_minor": 0}

Accuracy = 0.9101999998092651
Лучше на 0.001600027084350586

lstm_mean_last_concat_hd384_nl1_drop0.1 модель

```
{"model_id": "936b3cf82590480780b3d082d6b0b59b", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9192000031471252
Лучше на 0.009600043296813965

lstm_mean_hd384_nll_drop0.2 модель

```
{"model_id": "8fdc9bb417a142e289d27e1ee7c90f5c", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9187999963760376
Лучше на 0.009200036525726318

lstm_max_hd384_nll_drop0.2 модель

```
{"model_id": "7055c1ab6b2347c4a4b6f91ca39c8fd0", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9107999801635742
Лучше на 0.002200007438659668

lstm_mean_last_concat_hd384_nll_drop0.2 модель

```
{"model_id": "204f045c8c194220b7e909884553e2a6", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9192000031471252
Лучше на 0.009600043296813965

lstm_mean_hd384_nll_drop0.3 модель

```
{"model_id": "0fde16150cec493c9b049156fbd24cc8", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.918999969959259
Лучше на 0.009400010108947754

lstm_max_hd384_nll_drop0.3 модель

```
{"model_id": "e11f77c1b62045a3840d880f1adfc8b", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9139999747276306
Лучше на 0.0054000020027160645

lstm_mean_last_concat_hd384_nll_drop0.3 модель

```
{"model_id": "b49ce7ad1e6244dd92bf7d30375bbd26", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9165999889373779
Лучше на 0.00700002908706665

lstm_mean_hd384_nl2_drop0.0 модель

```
{"model_id": "ef3b643f13784d9dbc2f112840d0c3c3", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9205999970436096

Лучше на 0.01100003719329834

lstm_max_hd384_nl2_drop0.0 модель

```
{"model_id": "b944a05a8c3d4568ba70a5d2268de4c6", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9175999760627747

Лучше на 0.009000003337860107

lstm_mean_last_concat_hd384_nl2_drop0.0 модель

```
{"model_id": "cb1363fa234d4dde9661ec7a16ed057d", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9195999503135681

Лучше на 0.009999990463256836

lstm_mean_hd384_nl2_drop0.1 модель

```
{"model_id": "486c7468752e43a2b57f93333c82a1ca", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9157999753952026

Лучше на 0.006200015544891357

lstm_max_hd384_nl2_drop0.1 модель

```
{"model_id": "4bf5a06e81aa4feba157d7049fe0a695", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9175999760627747

Лучше на 0.009000003337860107

lstm_mean_last_concat_hd384_nl2_drop0.1 модель

```
{"model_id": "336592e143444757940536df9c7b87b6", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9165999889373779

Лучше на 0.00700002908706665

lstm_mean_hd384_nl2_drop0.2 модель

```
{"model_id": "5c60599e2a4b4465929b4c5c126f0696", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9156000018119812
Лучше на 0.006000041961669922

lstm_max_hd384_nl2_drop0.2 модель

{"model_id": "225a6b7c7bd24fe9be2c9c86aeb1524d", "version_major": 2, "version_minor": 0}

Accuracy = 0.9157999753952026
Лучше на 0.007200002670288086

lstm_mean_last_concat_hd384_nl2_drop0.2 модель

{"model_id": "d0e91fe556244f5788e67cda7c565432", "version_major": 2, "version_minor": 0}

Accuracy = 0.9193999767303467
Лучше на 0.0098000168800354

lstm_mean_hd384_nl2_drop0.3 модель

{"model_id": "fe686b25188448a80d73e78dd0a4a69", "version_major": 2, "version_minor": 0}

Accuracy = 0.9203999638557434
Лучше на 0.010800004005432129

lstm_max_hd384_nl2_drop0.3 модель

{"model_id": "31a600f2b6a646eba8b6a29e5648606b", "version_major": 2, "version_minor": 0}

Accuracy = 0.9195999503135681
Лучше на 0.010999977588653564

lstm_mean_last_concat_hd384_nl2_drop0.3 модель

{"model_id": "3d9c7e72171d4355afbfc8e5881f15f9", "version_major": 2, "version_minor": 0}

Accuracy = 0.9151999950408936
Лучше на 0.005600035190582275

lstm_mean_hd384_nl3_drop0.0 модель

{"model_id": "1d0e41a16e7f4ce7bf26596ac517badc", "version_major": 2, "version_minor": 0}

Accuracy = 0.9157999753952026
Лучше на 0.006200015544891357

lstm_max_hd384_nl3_drop0.0 модель

```
{"model_id": "d61fcc59c93c4c4b9a9231c2c8859fe8", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9161999821662903
Лучше на 0.007600009441375732

lstm_mean_last_concat_hd384_nl3_drop0.0 модель

```
{"model_id": "18057add8c8b4826b9ffb1fb821325f8", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9223999977111816
Лучше на 0.012800037860870361

lstm_mean_hd384_nl3_drop0.1 модель

```
{"model_id": "c18aa78023c344efa306e73f6c8135f6", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9205999970436096
Лучше на 0.01100003719329834

lstm_max_hd384_nl3_drop0.1 модель

```
{"model_id": "74f3a7e608c8410e9778dde346d2d27c", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.921999990940094
Лучше на 0.013400018215179443

lstm_mean_last_concat_hd384_nl3_drop0.1 модель

```
{"model_id": "0ab7b1c2fbb74a1b83b4a5ec430f2fba", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9174000024795532
Лучше на 0.007800042629241943

lstm_mean_hd384_nl3_drop0.2 модель

```
{"model_id": "b19a83631d49454e808163af1de78d5e", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9174000024795532
Лучше на 0.007800042629241943

lstm_max_hd384_nl3_drop0.2 модель

```
{"model_id": "f4b7c6085af849daef8fde94c6656eda", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.91839998960495
Лучше на 0.0098000168800354

lstm_mean_last_concat_hd384_nl3_drop0.2 модель

```
{"model_id": "0975aeb5ddb946eea77635ec38329583", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9217999577522278

Лучше на 0.012199997901916504

lstm_mean_hd384_nl3_drop0.3 модель

```
{"model_id": "42625a352a2b4d69a606abc2ca4d3aa8", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9217999577522278

Лучше на 0.012199997901916504

lstm_max_hd384_nl3_drop0.3 модель

```
{"model_id": "022442eef133456f913193905267cdb6", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9181999564170837

Лучше на 0.00959998369216919

lstm_mean_last_concat_hd384_nl3_drop0.3 модель

```
{"model_id": "3c446cdc91d549939bfff6bb5da0ae794", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9167999625205994

Лучше на 0.007200002670288086

lstm_mean_hd512_nl1_drop0.0 модель

```
{"model_id": "d2c33ed173a444429835a4ab8cafe700", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9147999882698059

Лучше на 0.005200028419494629

lstm_max_hd512_nl1_drop0.0 модель

```
{"model_id": "3eca93420ade49b296046ee3339c23b4", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9111999869346619

Лучше на 0.0026000142097473145

lstm_mean_last_concat_hd512_nl1_drop0.0 модель

```
{"model_id": "207e6080a1464137907769887e1f9e66", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9199999570846558
Лучше на 0.010399997234344482

lstm_mean_hd512_nll_drop0.1 модель

```
{"model_id": "dd5b9b8c84a446509d790b30035b3e19", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.91839998960495
Лучше на 0.008800029754638672

lstm_max_hd512_nll_drop0.1 модель

```
{"model_id": "776417a8b73a4dc9b18d49672f5b97d4", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9115999937057495
Лучше на 0.003000020980834961

lstm_mean_last_concat_hd512_nll_drop0.1 модель

```
{"model_id": "bef5a77bf5044efbab039d30830f5791", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9143999814987183
Лучше на 0.004800021648406982

lstm_mean_hd512_nll_drop0.2 модель

```
{"model_id": "427ed9f758fa46a4937058e7cef90d8b", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9167999625205994
Лучше на 0.007200002670288086

lstm_max_hd512_nll_drop0.2 модель

```
{"model_id": "e880c15024044a36bbe10ba8c387e658", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9177999496459961
Лучше на 0.009199976921081543

lstm_mean_last_concat_hd512_nll_drop0.2 модель

```
{"model_id": "ce71b11d035f4f97b6ee2561b5c210bd", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9205999970436096
Лучше на 0.01100003719329834

lstm_mean_hd512_nll_drop0.3 модель

```
{"model_id": "4de7ca1efd5a4a3bba0ac4308ba82948", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9175999760627747
Лучше на 0.008000016212463379

lstm_max_hd512_nl1_drop0.3 модель

```
{"model_id": "f905be3134614c26aa4b8f5d023bd84f", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.914199948310852
Лучше на 0.0055999755859375

lstm_mean_last_concat_hd512_nl1_drop0.3 модель

```
{"model_id": "5b9dadf0157a4629bdc27cc4abf67b38", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9175999760627747
Лучше на 0.008000016212463379

lstm_mean_hd512_nl2_drop0.0 модель

```
{"model_id": "51531d146ec447e6a620447ba7737b26", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9221999645233154
Лучше на 0.01260000467300415

lstm_max_hd512_nl2_drop0.0 модель

```
{"model_id": "c697a5be85984cf6b29410fdb7a7187b", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9129999876022339
Лучше на 0.004400014877319336

lstm_mean_last_concat_hd512_nl2_drop0.0 модель

```
{"model_id": "4470f66106144745a3530640b74fac02", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9156000018119812
Лучше на 0.006000041961669922

lstm_mean_hd512_nl2_drop0.1 модель

```
{"model_id": "d9a3f3a76d7c4611851993f765510d4d", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9181999564170837
Лучше на 0.008599996566772461

lstm_max_hd512_nl2_drop0.1 модель

```
{"model_id": "cce8e73c50dc4be4a713658ea4675a7a", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9157999753952026

Лучше на 0.007200002670288086

lstm_mean_last_concat_hd512_nl2_drop0.1 модель

```
{"model_id": "d43de5e7dde64779b5c8ba216e01535f", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9156000018119812

Лучше на 0.006000041961669922

lstm_mean_hd512_nl2_drop0.2 модель

```
{"model_id": "a4e2b18484a74b4099e1223c40a8fb00", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9167999625205994

Лучше на 0.007200002670288086

lstm_max_hd512_nl2_drop0.2 модель

```
{"model_id": "08b6b140520f4599a4fdf511f95bf530", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9167999625205994

Лучше на 0.008199989795684814

lstm_mean_last_concat_hd512_nl2_drop0.2 модель

```
{"model_id": "a333c8571902402494a3706e0d97b21c", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9199999570846558

Лучше на 0.010399997234344482

lstm_mean_hd512_nl2_drop0.3 модель

```
{"model_id": "7991d8e2b9e04a059b6b1cfea7898bbb", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9163999557495117

Лучше на 0.0067999958992004395

lstm_max_hd512_nl2_drop0.3 модель

```
{"model_id": "9b1dee1fe3bb48aa885e783fb9ce548e", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9175999760627747
Лучше на 0.009000003337860107

lstm_mean_last_concat_hd512_nl2_drop0.3 модель

{"model_id": "3c0165f4c98a40e8b0f7f04aaaf1e991", "version_major": 2, "version_minor": 0}

Accuracy = 0.9156000018119812
Лучше на 0.006000041961669922

lstm_mean_hd512_nl3_drop0.0 модель

{"model_id": "6cfff200a41a496cbd97e333b76433da", "version_major": 2, "version_minor": 0}

Accuracy = 0.9177999496459961
Лучше на 0.008199989795684814

lstm_max_hd512_nl3_drop0.0 модель

{"model_id": "abf633c9d4f9412b877c2c1dddc260f4", "version_major": 2, "version_minor": 0}

Accuracy = 0.9147999882698059
Лучше на 0.006200015544891357

lstm_mean_last_concat_hd512_nl3_drop0.0 модель

{"model_id": "dfb5a89bc1da465dbfc773ba755e51db", "version_major": 2, "version_minor": 0}

Accuracy = 0.920799970626831
Лучше на 0.011200010776519775

lstm_mean_hd512_nl3_drop0.1 модель

{"model_id": "3d51499d4f1c490bbaca5bb3d181191d", "version_major": 2, "version_minor": 0}

Accuracy = 0.9175999760627747
Лучше на 0.008000016212463379

lstm_max_hd512_nl3_drop0.1 модель

{"model_id": "627ac54a7ff8488385b9107006882c03", "version_major": 2, "version_minor": 0}

Accuracy = 0.9156000018119812
Лучше на 0.00700002908706665

lstm_mean_last_concat_hd512_nl3_drop0.1 модель

```
{"model_id": "3939190dd8674e72aafd5a3e3b127930", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.917199969291687
Лучше на 0.007600009441375732

lstm_mean_hd512_nl3_drop0.2 модель

```
{"model_id": "5de700c4b2af4d10b745f41467aefd5a", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9179999828338623
Лучше на 0.008400022983551025

lstm_max_hd512_nl3_drop0.2 модель

```
{"model_id": "ae47ba53388a439da6c00b44ebccdfd3", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9157999753952026
Лучше на 0.007200002670288086

lstm_mean_last_concat_hd512_nl3_drop0.2 модель

```
{"model_id": "ce602360a6f64dfe8abf8c834be97f5f", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9187999963760376
Лучше на 0.009200036525726318

lstm_mean_hd512_nl3_drop0.3 модель

```
{"model_id": "092c69a3023f4c75ab9858c054c40e0c", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9163999557495117
Лучше на 0.0067999958992004395

lstm_max_hd512_nl3_drop0.3 модель

```
{"model_id": "2d7f2ca23b274024b61edf28ec4f5427", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9185999631881714
Лучше на 0.009999990463256836

lstm_mean_last_concat_hd512_nl3_drop0.3 модель

```
{"model_id": "01b172e9620b418597e65e4eb9c184ca", "version_major": 2, "version_minor": 0}
```

Accuracy = 0.9161999821662903
Лучше на 0.006600022315979004

```
print(f'Лучшая accuracy: {best_acc}')
print(f'Лучшие параметры: {best_params}')
```

```
Лучшая accuracy: 0.9223999977111816
Лучшие параметры: {'hidden_dim': 384, 'aggregation_type':
'mean_last_concat', 'num_layers': 3, 'dropout_rate': 0.0}
```

```
Лучшая accuracy: 0.9223999977111816 Лучшие параметры: {'hidden_dim': 384,
'aggregation_type': 'mean_last_concat', 'num_layers': 3, 'dropout_rate': 0.0}
```

```
set_seed(SEED)

name_model = 'lstm'

num_epoch = 7

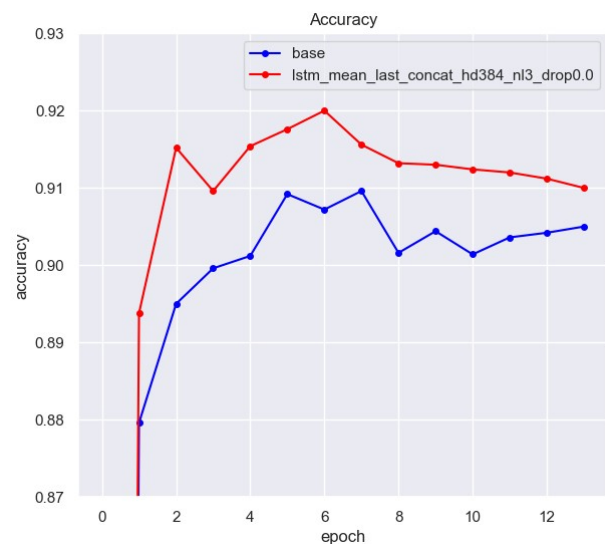
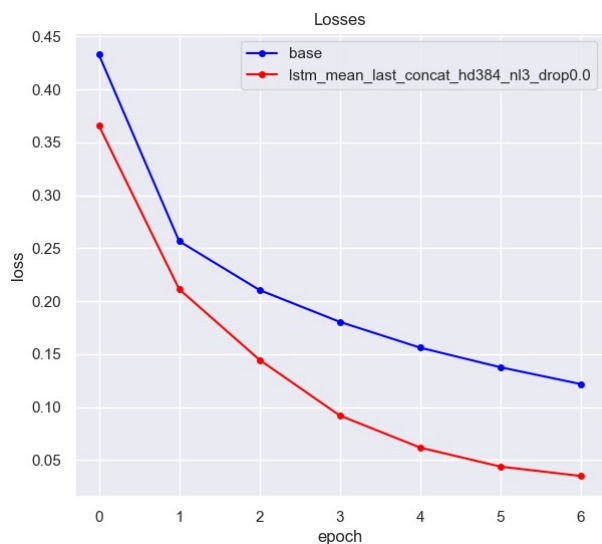
best_params = {'hidden_dim': 384, 'aggregation_type':
'mean_last_concat', 'num_layers': 3, 'dropout_rate': 0.0}

model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

agg_type = best_params['aggregation_type']

model_loss, model_acc = train_model(model, criterion, optimizer,

model_name=f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{
best_params["num_layers"]}_drop{best_params["dropout_rate"]}',
aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)
```



Accuracy = 0.9199999570846558
Лучше на 0.010399997234344482

Выбор оптимизатора

```
set_seed(SEED)

['adam', 'adamW_wd01', 'adamW_wd03', 'adamW_wd05']

num_epoch = 7

best_params = {'hidden_dim': 384, 'aggregation_type':
'mean_last_concat', 'num_layers': 3, 'dropout_rate': 0.0}
agg_type = best_params['aggregation_type']
name = 'adam'

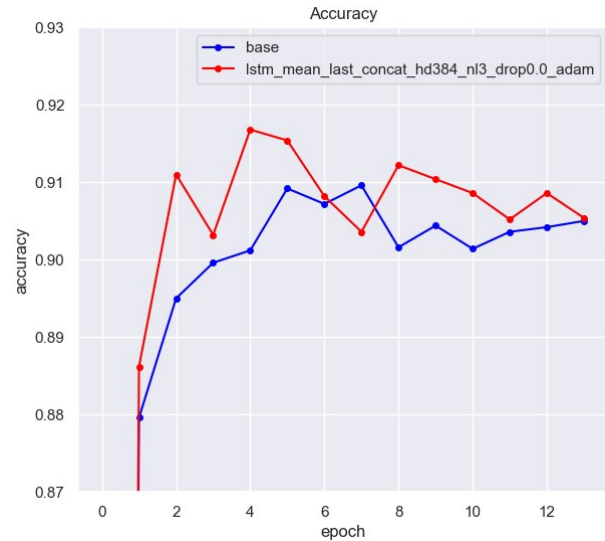
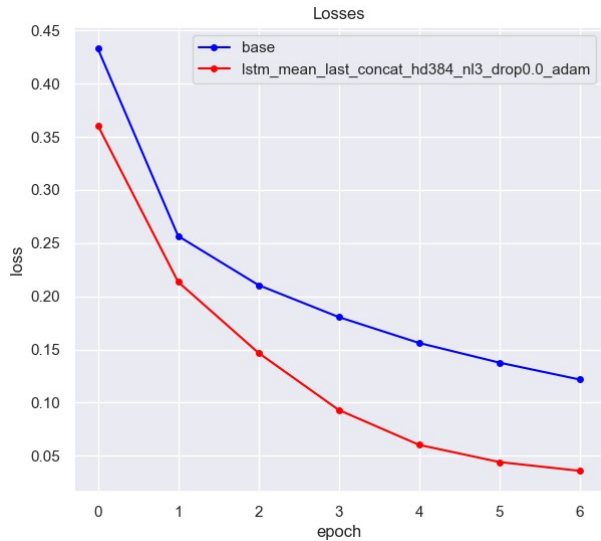
model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model.parameters())

best_acc = 0
best_optimizer_name = 'adam'
best_optimizer = optimizer

model_loss, model_acc = train_model(model, criterion, optimizer,

model_name=f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{
best_params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}',
            aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)

curr_acc = max(model_acc)
if curr_acc > best_acc:
    best_acc = curr_acc
    best_optimizer_name = name
    best_optimizer = optimizer
```



Accuracy = 0.9167999625205994
 Лучше на 0.007200002670288086

Adam заменим на AdamW и попробуем подобрать для него наилучшие гиперпараметры

```
set_seed(SEED)

['adam', 'adamW_wd01', 'adamW_wd03', 'adamW_wd05']

num_epoch = 7

best_params = {'hidden_dim': 384, 'aggregation_type':
'mean_last_concat', 'num_layers': 3, 'dropout_rate': 0.0}
agg_type = best_params['aggregation_type']
name = 'adamW_wd01'

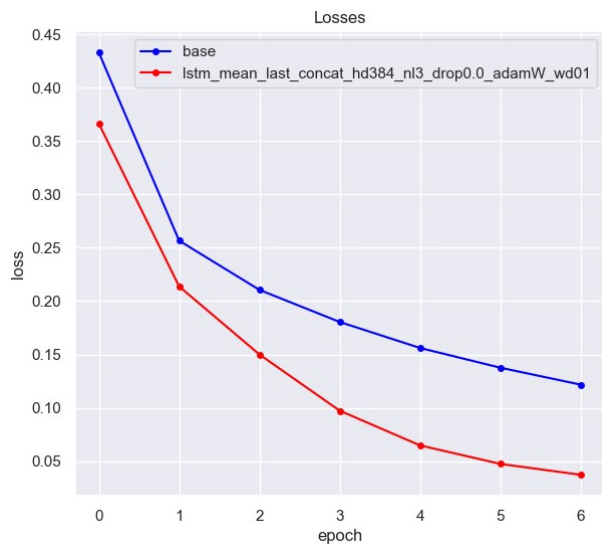
model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.AdamW(model.parameters(), lr=0.001,
weight_decay=0.01)

model_loss, model_acc = train_model(model, criterion, optimizer,

model_name=f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{
best_params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}',
aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)

curr_acc = max(model_acc)
if curr_acc > best_acc:
    best_acc = curr_acc
```

```
best_optimizer_name = name
best_optimizer = optimizer
```



Accuracy = 0.9161999821662903

Лучше на 0.006600022315979004

set_seed(SEED)

['adam', 'adamW_wd01', 'adamW_wd03', 'adamW_wd05']

num_epoch = 7

```
best_params = {'hidden_dim': 384, 'aggregation_type':
'mean_last_concat', 'num_layers': 3, 'dropout_rate': 0.0}
agg_type = best_params['aggregation_type']
name = 'adamW_wd03'
```

```
model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.AdamW(model.parameters(), lr=0.001,
weight_decay=0.03)
```

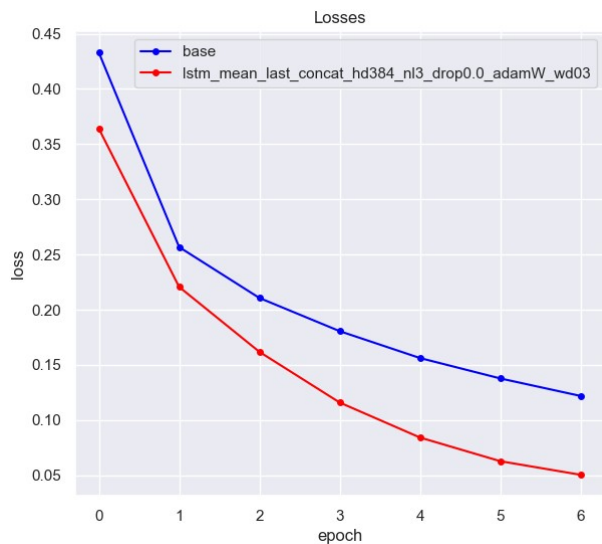
```
model_loss, model_acc = train_model(model, criterion, optimizer,
```

```
model_name=f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{
best_params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}',
aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)
```

```
curr_acc = max(model_acc)
```

```
if curr_acc > best_acc:
    best_acc = curr_acc
```

```
best_optimizer_name = name
best_optimizer = optimizer
```



```
Accuracy = 0.920199990272522
Лучше на 0.010600030422210693
```

```
set_seed(SEED)
```

```
['adam', 'adamW_wd01', 'adamW_wd03', 'adamW_wd05']
```

```
num_epoch = 7
```

```
best_params = {'hidden_dim': 384, 'aggregation_type':
'mean_last_concat', 'num_layers': 3, 'dropout_rate': 0.0}
agg_type = best_params['aggregation_type']
name = 'adamW_wd05'
```

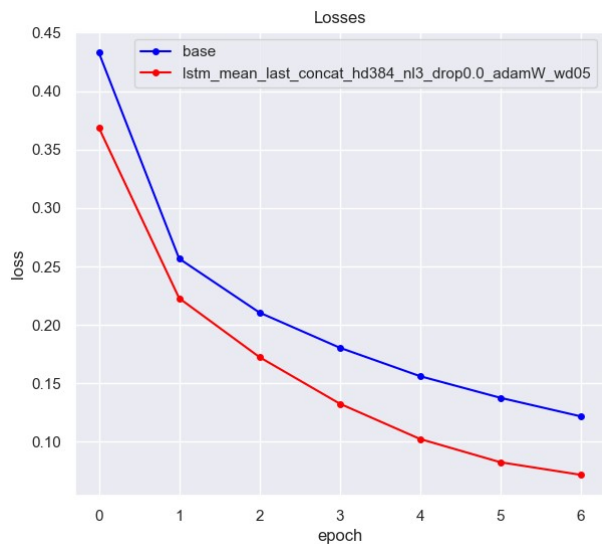
```
model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.AdamW(model.parameters(), lr=0.001,
weight_decay=0.05)
```

```
model_loss, model_acc = train_model(model, criterion, optimizer,
```

```
model_name=f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{
best_params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}',
aggregation_type=agg_type,
plot=True, num_epoch=num_epoch, base_mean_loss=base_mean_loss,
base_mean_acc=base_mean_acc)
```

```
curr_acc = max(model_acc)
if curr_acc > best_acc:
    best_acc = curr_acc
```

```
best_optimizer_name = name
best_optimizer = optimizer
```



Accuracy = 0.9193999767303467
Лучше на 0.0098000168800354

```
print(f'Лучшая accuracy: {best_acc}')
print(f'Лучший оптимайзер: {best_optimizer_name}')
```

Лучшая accuracy: 0.920199990272522
Лучший оптимайзер: adamW_wd03

Получение высокого качества (3 балла)

В конце вашей работы вы должны указать, какая из моделей дала лучший результат, и вывести качество, которое дает лучшая модель, с помощью функции `evaluate`. Ваша модель будет оцениваться по метрике `accuracy` следующим образом:

- $accuracy < 0.9$ --- 0 баллов;
- $0.9 \leq accuracy < 0.91$ --- 1 балл;
- $0.91 \leq accuracy < 0.915$ --- 2 балла;
- $0.915 \leq accuracy$ --- 3 балла.

Добавим в функцию обучения сохранение весов модели, с которой получается наилучшая точность. В конце обучения загрузим модель с именно с теми весами (на 'нужном' шаге обучения), с которыми получилась наилучшая точность.

```
def train_model_final(model, criterion, optimizer,
                      model_name='current',
                      num_epoch=7,
                      train_loader=train_dataloader,
                      test_loader=eval_dataloader):
```

```

eval_steps = len(train_loader) // 2

total_loss = []
total_acc = []

best_acc = 0.0
best_state = None

print(f'\n{model_name} модель')

for epoch in tqdm(range(num_epoch), desc=f'Training
{model_name}'):
    epoch_train_loss = 0

    model.train()
    for i, batch in enumerate(train_loader):    # for i, batch in
enumerate(tqdm(train_loader)):
        optimizer.zero_grad()
        logits = model(batch['input_ids'])
        loss = criterion(logits, batch['label'])
        loss.backward()
        optimizer.step()
        epoch_train_loss += loss.item()

        if i % eval_steps == 0:
            model.eval()
            curr_acc = evaluate(model, test_loader)
            total_acc.append(curr_acc)

            # Выбор лучших весов и их сохранение
            if curr_acc > best_acc:
                best_acc = curr_acc
                best_state = {k: v.cpu().clone() for k, v in
model.state_dict().items()}

            model.train()

    total_loss.append(epoch_train_loss / len(train_loader))

# Загрузка лучших весов
model.load_state_dict(best_state)

print(f'Accuracy = {max(total_acc)}')

return total_loss, total_acc

set_seed(SEED)

name_model = 'lstm'
best_params = {'hidden_dim': 384, 'aggregation_type':

```

```

'mean_last_concat', 'num_layers': 3, 'dropout_rate': 0.0}
agg_type = best_params['aggregation_type']
name = 'adamW_wd03'

num_epoch = 10

model = LSTMCharLM(**best_params, vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.AdamW(model.parameters(), lr=0.001,
weight_decay=0.03)

model_loss, model_acc = train_model_final(
    model, criterion, optimizer,

model_name=f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{
best_params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}',
    num_epoch=num_epoch
)

lstm_mean_last_concat_hd384_nl3_drop0.0_adamW_wd03 модель

{"model_id": "08956804a2ad44539b71f9d7004e3002", "version_major": 2, "version_minor": 0}

Accuracy = 0.918999969959259

print(f'Лучшая accuracy: {max(model_acc)}')
print(f'{name_model}_{agg_type}_hd{best_params["hidden_dim"]}_nl{best_params["num_layers"]}_drop{best_params["dropout_rate"]}_{name}')

Лучшая accuracy: 0.918999969959259
lstm_mean_last_concat_hd384_nl3_drop0.0_adamW_wd03

```

Метрика `accuracy`, полученная с помощью функции `evaluate`, которую дает лучшая модель.

Применим функцию `evaluate` отдельно от цикла обучения

```

model.eval()
final_accuracy = evaluate(model, eval_dataloader)
print(f'Accuracy = {final_accuracy}')

Accuracy = 0.918999969959259

```

$0.915 \leq 0.918999969959259$

Оформление отчета (2 балла)

В конце работы подробно опишите все проведенные эксперименты.

- Укажите, какие из экспериментов принесли улучшение, а какие --- нет.
- Проанализируйте графики сходимости моделей в проведенных экспериментах. Являются ли колебания качества обученных моделей существенными в зависимости от эпохи обучения, или же сходимость стабильная?
- Укажите, какая модель получилась оптимальной.
- Было опробовано: -- Агрегации ['max', 'mean', 'sum', 'max_norm', 'mean_norm', 'mean_last_concat', 'multi_concat']. Хорошо себя показали агрегации: 'max', 'mean', 'mean_last_concat', 'multi_concat'. -- Изменение num_layers и dropout_rate. Прирост точности был при num_layers=2, но был незначительный. -- Модели LSTM, GRU и Bidirectional RNN. Все они показали точность выше, чем Base model. Выбрал LSTM, так как у нее лучший показатель точности по отношению к моделям с другими архитектурами. -- Подбор гиперпараметров для модели LSTM. Подбор осуществлялся по гиперпараметрам с дальнейшим выбором лучшего набора: hidden_dims = [256, 384, 512] aggregation_types = ['mean', 'max', 'mean_last_concat'] num_layers_list = [1, 2, 3] dropout_rates = [0.0, 0.1, 0.2, 0.3]. -- Выбор оптимизатора Adam, AdamW с различными параметрами регуляризации.
- Лосс при проведении экспериментов стабильно уменьшался, выходил на плато примерно к 5 эпохе. Accurasy в большинстве случаев на валидационном датасете выходит на плато к 4-6 эпохе. Далее точность не изменяется, либо идет на спад. Дальнейшее увеличение количества эпох при обучении нежелательно, так как уже прослеживаются признаки переобучения модели.
- Модель LSTM с подобранными гиперпараметрами, агрегацией и оптимизатором ('hidden_dim': 384, 'aggregation_type': 'mean_last_concat', 'num_layers': 3, 'dropout_rate': 0.0, AdamW(lr=0.001, weight_decay=0.03)) показала лучший результат на валидации, который составил Accuracy ~ 0.92.

```
df_stats_sorted_acc = df_stats.sort_values('Max_accuracy',
ascending=False, ignore_index=True).copy()
display(df_stats_sorted_acc.head(10))
```

	Model	Min_loss	Max_accuracy	\
0	lstm_mean_last_concat_hd384_nl3_drop0.0	0.036661	0.9224	
1	lstm_mean_hd512_nl2_drop0.0	0.029818	0.9222	
2	lstm_max_hd384_nl3_drop0.1	0.045484	0.9220	
3	lstm_mean_hd384_nl3_drop0.3	0.045838	0.9218	
4	lstm_mean_last_concat_hd384_nl3_drop0.2	0.043594	0.9218	
5	lstm_mean_last_concat_hd512_nl3_drop0.0	0.034927	0.9208	
6	lstm_mean_hd384_nl3_drop0.1	0.039984	0.9206	
7	lstm_mean_last_concat_hd512_nl1_drop0.2	0.030039	0.9206	
8	lstm_mean_hd384_nl2_drop0.0	0.029248	0.9206	
9	lstm_mean_hd384_nl2_drop0.3	0.039431	0.9204	

	Diff_accuracy
0	0.0128
1	0.0126
2	0.0134
3	0.0122
4	0.0122
5	0.0112

6	0.0110
7	0.0110
8	0.0110
9	0.0108

```
df_stats_sorted_acc.to_csv('hw_2_model_stats_2026.csv', index=False)
```

	Model	Min_loss	Max_accuracy	Diff_accuracy
0	lstm_mean_last_concat_hd384_nl3_drop0.0	0.036661	0.9224	0.0128
1	lstm_mean_hd512_nl2_drop0.0	0.029818	0.9222	0.0126
2	lstm_max_hd384_nl3_drop0.1	0.045484	0.9220	0.0134
3	lstm_mean_hd384_nl3_drop0.3	0.045838	0.9218	0.0122
4	lstm_mean_last_concat_hd384_nl3_drop0.2	0.043594	0.9218	0.0122
5	lstm_mean_last_concat_hd512_nl3_drop0.0	0.034927	0.9208	0.0112
6	lstm_mean_hd384_nl3_drop0.1	0.039984	0.9206	0.0110
7	lstm_mean_last_concat_hd512_nl1_drop0.2	0.030039	0.9206	0.0110
8	lstm_mean_hd384_nl2_drop0.0	0.029248	0.9206	0.0110
9	lstm_mean_hd384_nl2_drop0.3	0.039431	0.9204	0.0108